

Better Manipulation for Robot Agents

System 0 Actions and Grounded Force–Position Control

Research Proposal I · Zimo Wen

September 2026

Abstract

Robot-RSI commonly expresses a manipulation goal as an end-effector target that an inverse-kinematics and trajectory backend converts into robot motion. This proposal asks when better manipulation requires a different action interface: a fast local policy that directly predicts robot actions, an explicit statement of desired interaction passed to a fixed feedback controller, or a learned local force policy conditioned on that statement. The research is grounded in specific contact mechanics rather than assuming that lower-level output is always superior. Force Policy already predicts interaction frames, force-control masks, reference wrenches, and motion chunks; these are prior capabilities, not proposed inventions. The intended contribution is a task-grounded interface that connects an agent’s interpretation of the scene to executable interaction requirements, with matched comparisons against direct-action and existing force-policy baselines. Success, contact quality, generalization, and cost jointly determine whether manipulation improves.

1. Problem Definition and Evidence from Existing Systems

What System 0 means here. We use System 0 operationally for a fast local sensorimotor policy below task reasoning, not as the name of a unique checkpoint or a claim that every such policy outputs torque. Its output might be joint position, joint velocity, EEF increments, or torque. These interfaces must be distinguished. ACT explicitly predicts future target joint positions [1]; Reactive Diffusion Policy uses a slow latent policy and a fast tactile-conditioned decoder [2]. Neither observation alone establishes arbitrary joint-torque control.

The current baseline. In the inspected Robot-RSI LIBERO path, EEF targets are passed through pose solving and servo execution to joint increments. This is a concrete target-plus-IK baseline. A direct joint-action policy can avoid a separate target-to-joint IK solve at its own output boundary, but it still needs a robot servo, limits, and dynamics. Conversely, predicting an EEF delta can still invoke IK or a Jacobian-based controller internally. Eliminating a named IK call is not itself a manipulation result.

What Force Policy already solves. Fang et al.’s Force Policy [3] predicts an interaction frame, a six-axis selection mask, reference wrench, and motion chunk. Its global vision and local force policies are asynchronously scheduled. The paper also uses task semantics and a VLM in demonstration-based interaction-frame recovery. Therefore, neither adding desired force to an output nor using language to help determine interaction structure can be claimed as new by itself.

A more specific hypothesis. An agent can improve manipulation by selecting and updating *scene-grounded interaction requirements*: which motion is intended, which contact should be maintained, in what frame, and within which task-supported force range. These requirements can drive fixed control or condition a learned local policy. The hypothesis is only supported if this improves over existing Force Policy and appropriately tuned controllers under matched information and data. Otherwise the result may be a useful action-interface comparison rather than a new control method.

Existing integration is not an experiment. A separate Robot-RSI force-control integration documents official ForcePolicy adapters, processors, a scheduler bridge, and hybrid-control streaming. Its recorded validation used fake SDKs and substitutes; it did not establish real checkpoint inference or hardware contact performance. This is more infrastructure than contact-stop alone, but it remains an implementation starting point rather than evidence for the proposed paper.

2. Approach A: Direct System 0 Action Prediction

Input and output. A slow agent supplies an instruction or bounded task-phase representation z_t . A local policy receives z_t , current images or visual features, proprioception, and a short sensor history. The primary direct-action variant predicts joint-position or velocity sequences. A position target can be executed by a fixed servo, illustratively $\tau = K_p(q^{\text{ref}} - q) - K_d\dot{q} + \tau_{\text{ff}}$, where the servo gains and feedforward terms belong to the declared backend. The language model does not generate each servo-cycle torque token.

What is being learned. Train the local mapping from the same physical demonstrations used by the EEF-target comparison, retaining actual commanded joint targets rather than substituting measured positions indiscriminately. A direct policy may learn contact-sensitive corrections and avoid poor target-to-joint choices, but it can also overfit embodiment and workspace geometry. Evaluate both ordinary manipulation and configurations where the existing pose solver struggles. Report solver failures separately from failures of the policy’s intended behavior.

Additional variants. Compare direct joint commands with a local EEF-increment head and, only when a torque interface and training setup are validated, a bounded torque or residual head. The EEF head is not labeled IK-free if its backend performs pose inversion. Keep sensing, action horizon, training data, and servo limits matched. The question is whether changing the executable interface improves success and contact behavior, not whether one interface is syntactically closer to a motor.

3. Approach B: Interaction Requirements Realized by Fixed Control

A grounded interaction contract. The agent emits $c_t = (\mathcal{F}_t, S_t, \mathcal{W}_t^{\text{ref}}, \xi_t^{\text{motion}}, \mathcal{B}_t, \mathcal{G}_t)$: an interaction frame, force/position axis mask, reference wrench, desired motion, calibrated bounds, and phase/termination conditions. The contract records the source of each physical quantity, such as a calibrated tool profile, measured demonstration, visible geometry estimate, or bounded development experiment. A phrase like “press gently” does not uniquely determine a force in newtons.

What the fixed program does. The program converts frames, validates units and signs, selects compatible motion and force axes, and runs a feedback controller. For a simple admittance realization, an illustrative velocity reference in one common frame is

$$\xi^{\text{cmd}} = (I - S)\xi^{\text{motion}} + SA(\mathcal{W}^{\text{ref}} - \mathcal{W}^{\text{meas}}),$$

with dimensioned translational/rotational admittance A . A vendor hybrid controller can instead implement this behavior internally. Robot dynamics, contact, saturation, and the motion backend still matter. There is no unique algebraic inversion from a desired force to a joint configuration; force is achieved through closed-loop interaction.

Physical interpretation. A desired wrench is different from measured wrench, a maximum-force stop condition, and impedance parameters. Express force and moment in a specified frame and at a specified origin; moving the wrench origin requires the appropriate moment transformation. Document whether the SDK represents force felt by the TCP or force exerted on the environment. Stiffness and damping may be fixed controller settings or additional declared outputs, but they are not interchangeable with the force-axis mask used by Force Policy.

Uncertainty. If geometry, force provenance, or the relevant task phase is uncertain, the agent must request an allowed observation or use a declared conservative calibration procedure. It must not invent a target force. All development identification and calibration costs are included when comparing with fixed or manually specified contracts.

4. Approach C: A Contract-Conditioned Local Force Policy

Method. Condition a learned local policy on the grounded contract, wrist observations, and force and proprioceptive history. It updates the interaction frame, mask, reference wrench, and local motion within declared task bounds. The motivation is that the agent specifies the intended physical outcome while the fast policy adapts to local contact. This reuses the established Force Policy representation; the research variable is the value and reliability of the explicit task-grounded conditioning and its updates.

Required comparison. Compare the official Force Policy architecture and labels with an otherwise matched contract-conditioned version, including a hand-specified-contract upper reference and a no-contract ablation. Preserve its processor, frame conventions, and task-specific labeling choices where applicable. Do not attribute gains from extra demonstrations, a different scheduler, or a stronger vision backbone to the contract. The authors report a 50 Hz local policy and unified scheduling; those are source-specific measurements, not achieved frequencies for our integration.

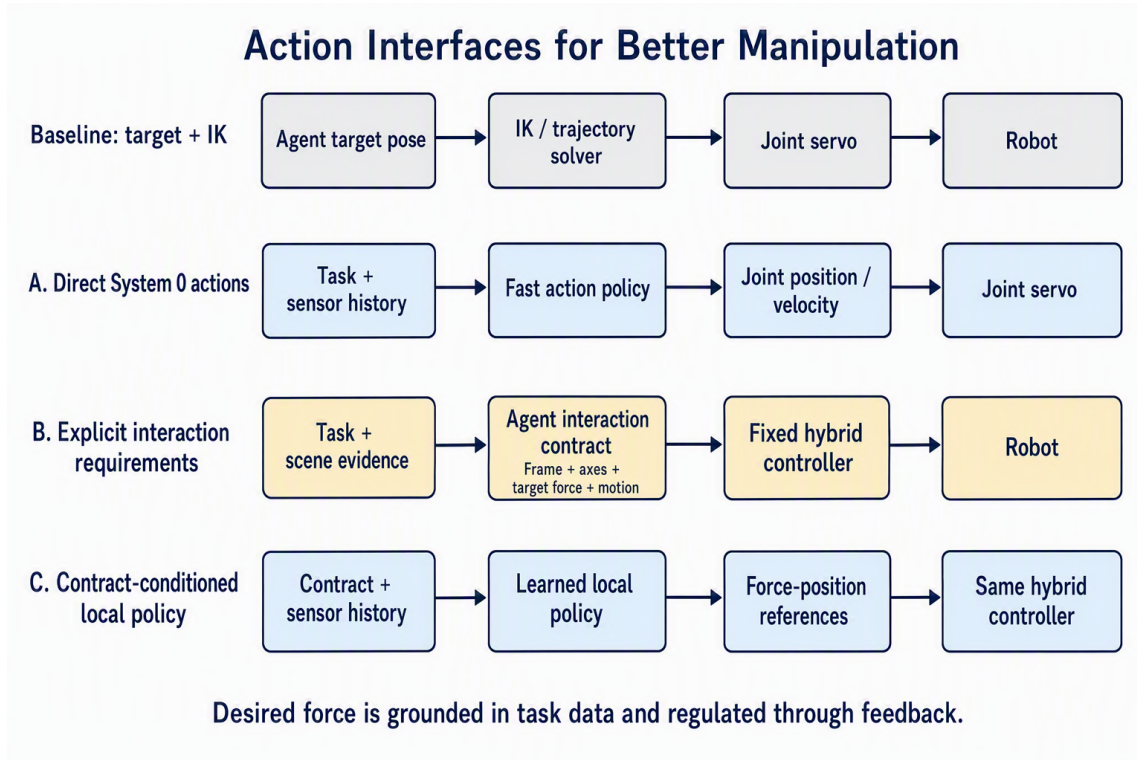


Figure 1: Target-plus-ik is the reference. A predicts direct local actions; B grounds interaction requirements for fixed feedback control; C conditions a learned local policy on those requirements. GPT-image-2 conceptual figure. Each backend and actual command space must be documented.

Why an agent could add value. Test task-phase or instruction changes that alter the desired interaction while preserving object appearance: maintaining contact versus withdrawing, or surface following versus a position-only approach. A contract can make this distinction explicit, but existing learned policies may already infer it. Only matched experiments on held-out task–geometry combinations can establish the agent’s additional contribution.

5. Concrete Scenarios and Experimental Design

Scenario	Appropriate interaction requirements	What must not be assumed
Surface following with a tool or pad	Track tangential motion, regulate normal contact, and specify tool orientation and force provenance.	The same normal force works for different surfaces, materials, or tools.
Constrained insertion	Advance along the identified insertion axis; permit declared lateral/angular compliance and cap axial resistance.	Higher insertion resistance should always be answered by larger constant pushing force.
Drawer or articulated sliding	Follow the estimated motion constraint; use lateral compliance and resistance/position feedback.	Drawer resistance is a universal desired axial-force target rather than a changing observation or limit.

Choosing force references. For surface following, derive an admissible normal-force range from development demonstrations and tool/material calibration, then select or predict a reference within that range. Report the task effect, such as sustained contact or achieved surface coverage, together with material-specific upper limits. For insertion, start with a declared low-speed motion and resistance limit; zero lateral wrench targets or an impedance policy may permit alignment only when the actual controller supports those semantics. Desired force is used only when the task truly requires it. No generic numeric force values are proposed as transferable robot settings.

Initial panel. Use four held-out object/geometry configurations in each of the three task families, three levels of relevant physical variation, and five initial seeds: 180 task episodes per method. Development and validation use distinct objects and task–geometry combinations. Test single changes before combinations: surface tilt, compliance, friction, insertion clearance, or articulated-axis error. Verify that the task remains physically solvable. These contact cells are declared extensions; standard LIBERO scores are reported separately.

Baselines. Include the current target-plus-IK path with tuned feedback; direct joint-action and local EEF-action policies; manually specified fixed hybrid control; the published Force Policy method where data and interfaces support it; agent-generated contracts with fixed control; and contract-conditioned local learning. Reactive Diffusion Policy provides an additional slow–fast sensorimotor comparison where its data requirements can be matched. Record the exact output space of every method instead of classifying all learned actions as torque control.

Data and hardware realism. Use synchronized images, commanded and measured joint/EEF motion, and calibrated wrench history. Force targets and interaction-frame labels need appropriate processing; a force sensor stream is not automatically an optimal reference trajectory. Force Policy’s public Push and Flip checkpoint is not a model for arbitrary insertion, scraping, or cutting. Obtain task-matched training data for comparisons. Hardware evaluation requires validated tools, frames, sensing, and control interfaces; no robot execution is part of preparing this proposal.

Standard task reporting. On the separate 120-task LIBERO short panel, report frozen task-level pass@10 with spatial/object/goal and task/object/swap/lan breakdowns. Keep original benchmarks, contact extensions, historical adaptive results, and trained-policy results distinct.

6. Planned Ablations and Evidence for Better Manipulation

Experiment A: action space versus controller quality. Compare target-plus-IK, direct joint actions, and local EEf actions using the same demonstrations, sensing, and comparable model capacity. Tune the IK baseline and disclose the trajectory and servo settings. Evaluate task completion, contact disturbances, and motion quality, including held-out configurations. A direct policy beating an untuned solver does not establish a general action-representation advantage.

Experiment B: desired contact versus force limits. Compare position control with force sensing, contact-stop behavior, fixed hybrid references, and task-grounded references. All methods observe the same wrench signal. Separately ablate frame estimation, axis selection, desired wrench, and motion intent. This distinguishes the value of force feedback from the value of an explicit target. A clamp that prevents large forces is not evidence that the intended force was tracked.

Experiment C: what the agent contributes beyond Force Policy. Compare no contract, fixed human-specified contracts, generated contracts, and generated contracts updated after observable contact changes. Match training labels, local network, and scheduler. Test instruction/phase changes and new task–geometry combinations while holding the scene appearance approximately constant where possible. Add incorrect-frame, incorrect-mask, and unsupported-reference diagnostics only offline in simulation and label them clearly. If existing Force Policy matches performance, limit the novelty claim to the measured task interface or generalization setting.

Experiment D: grounding and calibration. Compare references obtained from task-matched demonstrations, calibrated ranges, and language-only numeric proposals as a simulation diagnostic. The last condition tests the cost of ungrounded specification, not an endorsed deployment strategy. Measure whether the proposed contract predicts a useful physical quantity and whether its provenance supports transfer to a new object. Calibration trials and extra sensing are counted as resources.

Metrics. Final simulator/harness success is primary. During defined contact phases, measure reference-force RMSE, contact-loss duration, peak force, impulse, and task-specific physical effects. Report pose error and settling behavior in motion phases. Force tracking alone can be perfect for a useless reference, so it must be paired with actual manipulation success. An insertion that reaches high force without insertion progress is a failure, not successful force regulation.

Observed pattern	Supported interpretation
Direct actions help pose-solver-sensitive cases only	A useful embodiment/interface boundary, not universal superiority of lower-level control.
Grounded references improve contact and task success	Supports task-specific interaction specification, after matching feedback access.
Force tracking improves but task success does not	The chosen physical objective may be inappropriate.
Contract conditioning matches unmodified Force Policy	No incremental learned-control contribution has been established.
Manual contracts work but generated ones fail	Grounding the agent’s physical specification is the bottleneck.

Use paired seeds and task-clustered intervals, with multiple training seeds for learned variants. Report inference, calibration, training, and execution costs separately. Additional samples are determined from validation variance; the proposed panel is not an assertion of sufficient statistical power.

7. Implementation Boundaries and Expected Contribution

Reuse before invention. The inspected Force Policy code and paper already expose action/frame/wrench/mask outputs, task-specific processors, and a slow-fast scheduler. The Robot-RSI integration documents frame conversion and fixed SDK realization. Reuse these interfaces as declared baselines rather than rebrand them. The new paper must identify a measured gain from task-grounded specification, direct local action learning, or their combination; these are hypotheses, not established capabilities.

Control facts that must remain explicit. Joint targets still require a servo. EEF targets may still require IK. A desired wrench needs a contact model or feedback realization, not a unique inverse solution. Impedance sets a force-motion relationship, while hybrid control selects which axes regulate motion or force. Wrench frame, origin, sign, units, and sensor compensation are part of the method. SDK command acceptance, completed streaming duration, or a fake-interface test does not establish contact convergence or task success.

Scope of the agent and the local policy. The agent selects task phases and grounds interaction requirements on a slower timescale. The local policy or fixed feedback program performs sensor-conditioned execution. Actual update rates and delays must be measured. We do not claim that GPT emits reliable high-frequency torques. System 0 is the functional placement of the local sensorimotor module, whose action representation and training are specified separately for each variant.

Evidence and failure accounting. Final outcomes come only from the simulator or independent harness. Exclude provider, transport, image, resource, and interrupted records from valid-verdict success denominators and report them separately, with implementation errors and coverage. Retain force curves, videos, rejected specifications, failed runs, and exact manifests. Report tokens, VLM and unmetered calls, wall time, and perception/action/recovery time. Learned models require matched and held-out simulation evaluation; training loss does not establish manipulation improvement.

Expected paper contribution. The strongest plausible result is an empirically supported interface between task reasoning and physical interaction: the agent specifies grounded, scene-appropriate motion and contact requirements, and an explicitly defined local realization executes them better under selected generalization conditions. A useful negative result is that direct actions or published Force Policy already suffice. Either outcome must explain concrete manipulation behavior rather than equating lower abstraction with greater intelligence. The second proposal separately studies generated reactive policy structure; the third retains GPT to audit and correct VLA actions.

References

- [1] Zhao et al. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware (ACT). 2023. <https://arxiv.org/abs/2304.13705>
- [2] Xue et al. Reactive Diffusion Policy: Slow-Fast Visual-Tactile Policy Learning for Contact-Rich Manipulation. 2025. <https://arxiv.org/abs/2503.02881>
- [3] Fang et al. Force Policy: Learning Hybrid Force-Position Control Policy under Interaction Frame for Contact-Rich Manipulation. 2026. <https://arxiv.org/abs/2602.22088>
- [4] Force Policy official source, model heads, processors, and scheduler. <https://github.com/force-policy/ForcePolicy>
- [5] Code as Policies: Language Model Programs for Embodied Control. 2022. <https://arxiv.org/abs/2209.07753>
- [6] Noematrix Team. RoboRSI research preview. 2026. <https://lab.noematrix.ai/blog/2-roborsi-research-preview/>

Robot Agents That Generate Reactive Policies

State Machines, Stateful Programs, and Constraint-Based Execution

Research Proposal II · Zimo Wen

September 2026

Abstract

A robot agent may write an executable plan that becomes obsolete while the robot is acting. Repeated language-model calls can update the plan, but they place inference latency inside the response path. A manually written collection of if–else rules avoids that delay without demonstrating an agent-specific research contribution. This proposal instead asks whether an agent can synthesize a task-appropriate reactive policy whose local execution continuously incorporates new observations. Three representations are compared: a generated state machine, a stateful feedback program, and a generated objective/constraint specification solved locally. All use the same sensors and motion backend. The paper tests whether the generated feedback structure transfers to unseen tasks and changes, reduces unnecessary replanning, and improves actual recovery at a matched computation budget. State machines are one candidate representation within a broader policy-synthesis problem, rather than the full definition of responsiveness.

1. Research Question: Generate a Policy, Not Only an Action List

A motivating difference. An open-loop plan says “move to location A, descend, close the gripper.” A reactive policy says “track the current cup estimate, descend when alignment is sufficient, and close only when the grasp condition is observed.” If the cup moves slightly, local execution updates its target without waiting for another complete agent turn. If the cup disappears or the task changes, the policy identifies an invalid assumption and escalates. The distinction concerns the dependence of actions on observations, not the number of if–else statements.

Agent-specific contribution. A fast controller or a background thread can be useful without being a new Robot Agent method. Here the agent must infer what needs monitoring, which observations change the current behavior, what memory is required, and when its local policy stops being applicable. The generated object is a reusable feedback mapping, not simply a fixed sequence with a more frequent timer. Its value is evaluated on task and disturbance combinations not encoded by a developer in advance.

Problem formulation. A synthesis model maps instruction g , initial permitted observations, and a declared primitive library to a policy P . At local execution step t , P maps observation o_t and memory z_t to an action reference, updated memory, and an explicit status: continue, complete, or escalate. The low-level motion realization is fixed across comparisons. This separates policy representation from the first proposal’s physical action interfaces and from the third proposal’s GPT audit of a VLA candidate.

Existing work. Code as Policies already generates programs that may include feedback. Real-Time Chunking addresses asynchronous execution of learned action chunks; Reactive Diffusion Policy and Force Policy provide slow–fast contact architectures. We cannot claim novelty for feedback loops or asynchronous execution alone. The proposed question is whether *task-conditioned synthesis of feedback structure and applicability conditions* produces better generalization and response than open-loop programs, fixed generic monitors, and developer-written state machines.

Hypotheses. H1: generated task-specific feedback improves disturbed-task completion over fixed sequences at matched sensing and action access. H2: state machines, stateful programs, and constraint policies have different strengths across discrete mode changes, continuous tracking, and geometric replanning. H3: explicit validity and escalation conditions reduce costly full replanning while preserving recovery. If a generic handcrafted monitor performs equally well, the agent-specific claim must be narrowed.

2. Approach A: Agent-Generated State Machines

Policy representation. Represent a policy as states, state-local actions, observable guards, and transitions. For grasping, possible states are observe, approach, align, grasp, lift, and recover. The agent chooses this structure from the task, rather than merely filling in a fixed task-specific graph. A transition might require stable alignment for several observations; recovery might depend on an observed grasp inconsistency. Store timers and short measurement histories where necessary to avoid switching on one noisy frame.

Generation contract. Each guard names its observation source, freshness requirement, threshold units, and action consequence. States specify entry behavior, local update, timeout, and exit. The runtime checks missing transitions, unreachable states, loops without bounded progress, incompatible action ownership, and absent observations. These checks establish structural executability, not task success. Guards cannot call a hidden simulator success predicate or use exact target geometry unavailable through sensors.

Local execution and repair. The state machine consumes a stream of code-backed perception and proprioception at a fixed measured rate. It issues short-horizon references through the common motion backend. An invalid assumption triggers a bounded agent revision of the relevant state or transition; unaffected states are retained. Test-time revision remains within the current episode and is logged separately from development-time skill accumulation. Compare generated graphs with fixed generic graphs and task-specific human graphs.

3. Approach B: General Stateful Feedback Programs

Beyond discrete branches. Some behavior is naturally continuous or depends on history. A program can estimate target velocity, smooth noisy observations, maintain object identity through brief occlusion, and continuously update an approach offset. It need not enumerate every possible target location as a separate state. The agent generates a local function with the common contract

$$(u_t, z_{t+1}, r_t) = P(o_t, z_t), \quad r_t \in \{\text{continue, complete, escalate}\}.$$

Here u_t is a motion reference and z_t stores permitted estimates and timers. The discrete status describes execution management; the action mapping itself can be continuous and stateful.

Example. A tracking policy computes the current grasp approach from the latest observed object pose and a task-specific offset, updates a filtered velocity estimate, and reduces progress when uncertainty rises. It transitions toward grasping only when measured alignment and observation freshness satisfy the policy. Persistent loss of the target produces escalation. The robot does not chase a coordinate frozen at program-generation time.

Implementation boundary. Programs use declared geometry, filtering, perception, and motion functions in a restricted runtime. No arbitrary filesystem access, simulator introspection, or unbounded remote model calls occur inside the local step. Bound memory, step duration, output magnitude, and revision count. If the backend only offers a blocking full-trajectory call, expose a validated short-step interface before claiming continuous response; adding a Python loop around blocking calls does not establish the required behavior.

Comparisons. Remove memory, freeze target updates, and replace task-conditioned logic with a generic tracker while preserving the same data stream. Compare full regeneration with local patching. If performance comes entirely from a stronger perception module or more frequent observations, the program-synthesis contribution has not been isolated.

4. Approach C: Generated Objectives and Constraints

Representation. The agent generates a task-specific cost and observable constraints, then a fixed local solver updates actions as the scene changes. For example, reaching toward a moving target can balance current goal error and motion smoothness while respecting estimated obstacle clearance. A short-horizon controller repeatedly solves

$$\min_{u_{t:t+H}} C_P(\hat{s}_t, u_{t:t+H}) \quad \text{subject to} \quad g_P(\hat{s}_t, u_{t:t+H}) \leq 0,$$

where $\hat{s}_t$ is estimated from permitted observations. The agent determines the task-dependent objectives, constraint sources, and applicability conditions; the solver supplies numerical optimization. No exact hidden obstacle pose enters the constraints.

Why this is a broader policy. New target and obstacle estimates change the optimization directly, without requiring a new if-else branch for every layout. Discrete stage changes can be represented by a small supervisor. Infeasibility, stale observations, or an invalid local dynamics approximation trigger escalation. A complex solver is not assumed superior; compare simple analytic tracking and a fixed generic objective under matched runtime budgets.

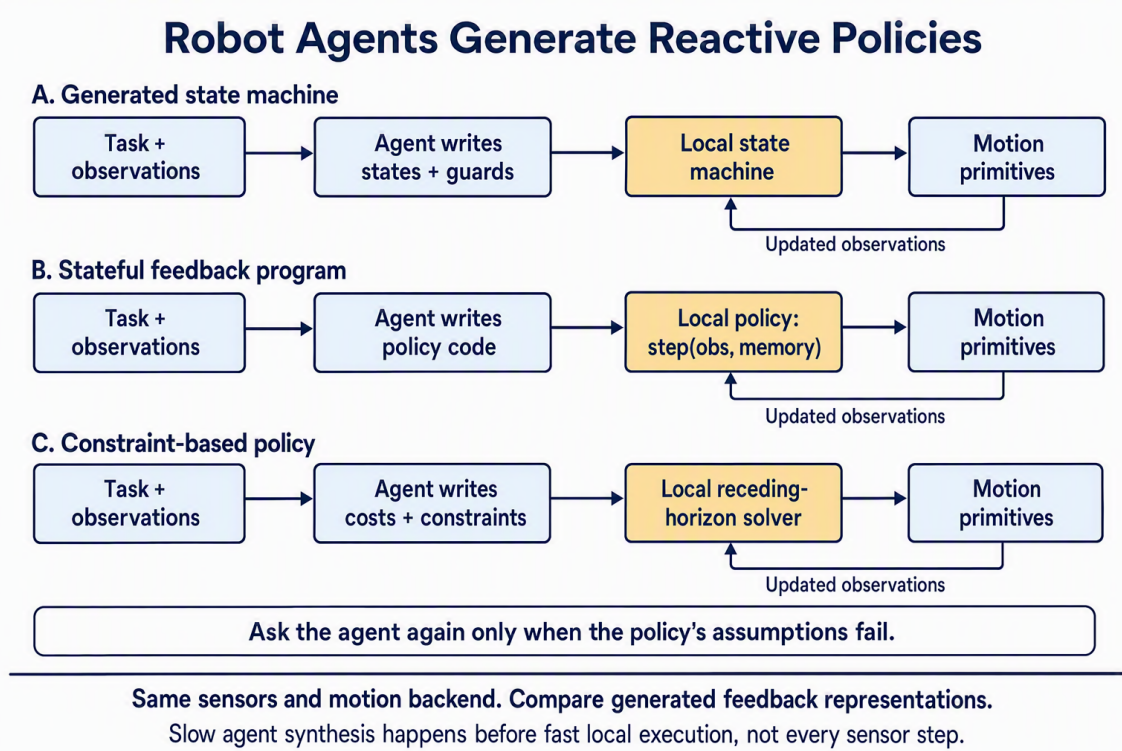


Figure 1: The agent synthesizes one of three feedback representations; updated observations enter its local execution loop. The sensor stream does not require a new language-model call at every step. GPT-image-2 conceptual diagram. All methods share perception and motion primitives.

Common policy lifecycle. Synthesize, validate, initialize memory, execute local updates, check completion or applicability, and request a bounded revision if needed. Record the generated policy version and the observation that justified each revision. Replacing a policy requires a controlled handoff; an obsolete asynchronous result cannot overwrite a newer policy. This lifecycle is shared infrastructure, not three separate higher-performance controllers.

5. Experimental Design: Generalization of Reactive Structure

Task and disturbance families. Use LIBERO short tasks with matched disturbances: target movement during approach, transient occlusion, changed visible path clearance, grasp inconsistency, and instruction or stage changes. Disturbances remain physically meaningful and leave the task solvable. The test set includes unperturbed tasks to measure regressions caused by unnecessary feedback and escalation. Hidden event schedules stay with the evaluator.

Task-level splitting. Group near-duplicate recipes and their language/layout variants before creating development, validation, and test splits. The synthesis model sees the test instruction and initial observations, but does not receive hand-coded rules for its event sequence. Hold out both tasks and task–disturbance combinations. Report whether the primitive library already contains a full task solution; otherwise retrieval of that skill could masquerade as new reactive synthesis.

Proposed panel. Begin with 24 held-out task–disturbance pairs, three strengths, and five seeds: 360 episodes per method. An additional matched subset varies observation or model latency. Determine the actual delay levels from validation relative to the task timescale, rather than copying arbitrary millisecond targets. Task pairs sharing a recipe remain one statistical cluster. The panel size is provisional and must be assessed against development variance.

Comparison	What is generated or fixed	Question isolated
Open-loop program	Fixed action sequence	Value of observation dependence
Repeated agent replanning	New plan at fixed intervals	Response benefit versus inference cost
Generic manual monitor	Fixed event rules and recovery	Value of task-specific synthesis
Task-specific manual FSM	Human-designed states and guards	Strong structural reference
Generated FSM	States, guards, and transitions	Discrete feedback synthesis
Stateful policy program	Continuous actions and memory	Broader feedback representation
Constraint-based policy	Task costs and constraints	Value of local optimization

Fairness. All methods use identical perception outputs, timestamps, short-horizon motion commands, and motion-controller limits. Match local execution rates and separately compare fixed total VLM-call or token budgets. A manual state machine may require more human design effort, which should be documented separately rather than converted into an arbitrary performance penalty. Do not give generated policies additional hidden sensing or stronger task primitives.

Two evaluations. A policy-level diagnostic injects observations into frozen generated policies to inspect guard decisions, memory behavior, and output validity. It does not count as robot task success. Full closed-loop simulation then measures execution under actual changing observations. The standard 120-task LIBERO panel separately reports frozen task-level pass@10 by spatial/object/goal and task/object/swap/lan, without mixing dynamic-extension results into the headline.

6. Planned Ablations and Meaningful Response Metrics

Experiment A: generated feedback versus more rules. Compare generic manual rules, task-specific manual FSMs, and generated policies with a fixed primitive set. Hold code size or synthesis-token budget approximately controlled in a secondary panel, but do not use the number of branches as a capability metric. Test whether generated guards monitor the right task-dependent quantities and whether the resulting correction helps after the event. More conditions can increase false triggers rather than improve response.

Experiment B: discrete, stateful, and optimization-based policies. Evaluate mode changes, continuous target drift, brief occlusion, and changing obstacle geometry separately. Expected strengths are hypotheses: FSMs may suit discrete stages, stateful programs may preserve tracking through ambiguous observations, and constraints may help spatial adjustment. Test these claims rather than selecting only tasks favorable to each representation. Report when a simpler representation achieves equal success at lower synthesis or runtime cost.

Experiment C: memory and applicability. Remove history, smoothing, freshness checks, and escalation conditions separately. Compare local patching with regeneration of the entire policy. Measure whether the agent recognizes an invalid assumption and revises the responsible component, or repeatedly changes unrelated code. Test-time policy versions and their triggers are retained. Failed patches and excessive escalations count against task and resource budgets.

Experiment D: response under delay. Measure the event time, first observation containing it, local policy update, controller acceptance, and actual corrective actuation. Local policies should absorb changes within their declared validity domain without waiting for an agent call. Out-of-domain events require slower reasoning and must be reported separately. Compare with asynchronous action-chunk execution where appropriate, but do not attribute a scheduler’s existing advantage to policy synthesis.

Primary metrics. Report final disturbed-task success, harmful response on unperturbed tasks, false/missed triggers, intervention count, escalation rate, response p50/p95, recovery time, and VLM cost. A program detecting the event but failing to change the robot is not responsive. A program that stops instantly but never resumes has a stop-path benefit, not successful manipulation recovery. Nonresponses remain visible with their timeout rate and censoring, rather than disappearing from latency summaries.

Possible result	Supported conclusion
Generated policies generalize and reduce replanning cost	Supports synthesizing task-dependent feedback structure.
Manual generic rules match performance	The agent-specific contribution is limited in this task set.
Stateful programs help only during occlusion	Evidence for memory in a specific observable regime.
Constraint policies reduce geometric failures but cost more time	A representation–compute tradeoff, not uniform superiority.
More code increases false escalation and regressions	Feedback complexity requires validity and relevance checks.

Use matched seeds, task-clustered confidence intervals, and multiple synthesis samples where generation is stochastic. Freeze prompts and selection rules before testing. Select checkpoints or programs using validation only; success-conditioned cherry-picking of generated code cannot establish deployment reliability.

7. Feasibility, Risks, and Expected Paper Contribution

Robot-RSI integration. The Planner or Engineer generates the policy representation, the runtime validates and executes it over existing perception and motion primitives, and visible traces support bounded revision. The proposal does not require a newly learned force controller. Existing implementations have different cancellation and streaming capabilities, so document the selected backend and hold it fixed across methods. A lock fix or faster transport alone is infrastructure work, not the policy-synthesis contribution.

Limits of verification. A syntactically valid state machine can contain incorrect guards; a terminating program can use an inappropriate recovery; an optimizer can satisfy a badly specified objective. Structural checks must be complemented by actual simulator outcomes. Robot observations are imperfect, so policies need a stated uncertainty/freshness domain. Neither handcrafted guards nor agent-generated constraints are presumed to guarantee safety or task completion.

Preventing hidden-state leakage. The agent, local code, and reviewers receive only declared images, estimated geometry, proprioception, and tool receipts. Hidden task predicates, future events, object ground truth, and private evaluator checkpoints are inaccessible. Learning a guard from a visible signal is allowed; reading the simulator’s answer to decide a transition is not. Post-hoc adjudication remains independent of the generated program’s own completion flag.

Evidence and cost. Preserve generated policies, all revisions, videos, trajectories, failures, and exact manifests. Count success only from final simulator/harness verdicts, with provider, transport, image, resource, and interrupted records reported separately outside valid-verdict denominators. Also show implementation errors, invalid-program rates, and incomplete coverage. Record synthesis and revision tokens, VLM/unmetered calls, wall time, perception/action/recovery time, local-step latency, and solver cost. Efficiency claims require task success to remain acceptable, not merely fewer language-model calls.

Expected contribution. The paper should establish whether a robot agent can turn task understanding into a feedback policy that continues to work as observations change. Its methods compare a finite-state representation with more general stateful and constraint-based policies; its experiments isolate generation from stronger sensing and control. The central result is generalization of the reactive structure under a fixed execution backend. This is different from the first paper’s action/force representation and the third paper’s retained GPT audit of VLA proposals.

Scope of the claims. Feedback programs, hybrid systems, local optimization, and asynchronous policies already exist. A new claim must concern how the agent synthesizes, grounds, and revises them, and what measurable benefit this yields. If fixed manual policies suffice, report that outcome. The proposal contains experimental designs, not newly measured success rates or promises of hard real-time execution.

References

- [1] Code as Policies: Language Model Programs for Embodied Control. 2022. <https://arxiv.org/abs/2209.07753>
- [2] Real-Time Execution of Action Chunking Flow Policies. 2025. <https://arxiv.org/abs/2506.07339>
- [3] Reactive Diffusion Policy: Slow-Fast Visual-Tactile Policy Learning for Contact-Rich Manipulation. 2025. <https://arxiv.org/abs/2503.02881>
- [4] Force Policy: Learning Hybrid Force-Position Control Policy under Interaction Frame for Contact-Rich Manipulation. 2026. <https://arxiv.org/abs/2602.22088>
- [5] Noematrix Team. RoboRSI research preview. 2026. <https://lab.noematrix.ai/blog/2-roborsi-research-preview/>

GPT-Reviewed VLA

Keep the Policy or Correct Its End-Effector Action

Research Proposal III · Zimo Wen

September 2026

Abstract

A vision–language–action policy can execute familiar manipulation efficiently, while a general-purpose GPT model can inspect task context and identify errors that the policy may miss. This proposal studies a deployed hybrid agent that retains both components. The VLA proposes an action chunk, and GPT chooses between two execution branches: keep the original action, or construct a bounded end-effector (EEF) correction through Code as Policy and audit it before execution. The same GPT model can perform selection, correction, and review in separate calls. The central question is whether selective correction improves task success without damaging actions the VLA would already execute correctly. We also propose a benchmark with natural rollouts and private, paired checkpoint continuations to separate branch selection, correction quality, unnecessary intervention, and inference cost. GPT remains active during evaluation and deployment; removing it is not the objective.

1. Research Question and the Intended Paper

Deployment mechanism. A VLA proposes behavior from an instruction and current observations. GPT reviews this proposal and the visible scene. If it is appropriate, execution retains the original action. If a local geometric change is needed, GPT writes an EEF correction, reviews that correction, and submits the approved result. After execution, new observations re-enter the loop. This is a persistent review-and-correction system, rather than a training-only teacher whose role disappears after fine-tuning.

Why two branches matter. A reviewer that always intervenes can damage an already capable VLA. A reviewer that always accepts preserves the VLA but cannot rescue mistakes. For example, the proposed grasp may already align with a cup, or it may need a small lateral or orientation adjustment. The research problem is to identify the cases where preserving the proposal is preferable and to generate a useful correction when it is not. Decision quality and correction quality must both be measured.

EEF correction, not policy-weight modification. Here EEF denotes end-effector pose or motion commands. GPT changes a bounded part of the current action trajectory, using explicit position/orientation offsets, local waypoints, or a short feedback program. The main study freezes VLA weights and retains existing low-level controllers. GPT does not introduce a learned force controller or update the VLA’s parameters at each review. The effect on the executed policy comes from selecting and modifying actions at runtime.

Central hypothesis. A persistent GPT reviewer can improve a frozen VLA’s closed-loop success by applying a small number of useful EEF corrections, while retaining most actions that are already effective. The paper should show a favorable success–intervention–latency tradeoff compared with always keeping, always correcting, and matched heuristic gates. It should also identify conditions in which review or self-audit is unreliable.

Relationship to the other proposals and prior work. The first paper studies which control quantities the agent should output; the second studies when changed observations can affect execution. This paper studies **which VLA proposals to trust and how to correct the selected ones**. Code as Policies provides executable programs, while VLA policies provide nominal actions. DAgger-style aggregation is a possible later extension, but is not the main deployment mechanism or evaluation target. A broader literature comparison is needed before asserting a first-of-its-kind contribution.

2. Method: Binary Review with an Audited EEF Correction

Visible input. Let $A_t = \pi_\theta(o_t, g, h_t)$ be the VLA’s proposed short action sequence. GPT receives the instruction, allowed images and proprioception, bounded visible history, and an inspectable representation of A_t . Where a VLA outputs joint commands, a declared adapter may express their intended EEF motion using robot kinematics. It must not inspect hidden object poses or simulator contact truth. The candidate remains associated with its observation timestamp, action version, and coordinate convention.

Binary execution rule. The reviewer produces $b_t \in \{\text{KEEP}, \text{CORRECT}\}$ and a concise, observable justification. For an approved correction program p_t :

$$A_t^{\text{exec}} = \begin{cases} A_t, & b_t = \text{KEEP}, \\ C(A_t, p_t, o_t), & b_t = \text{CORRECT}. \end{cases}$$

C applies an explicitly bounded EEF edit and translates it to the existing action interface. KEEP means the unchanged valid proposal, not a newly generated replacement trajectory. CORRECT edits the declared segment and records what changed. These remain the two task-action branches; waiting for a fresh observation or reporting an invalid proposal is execution management, not an undisclosed third controller.

Correction and audit. In the CORRECT branch, GPT creates a candidate containing a frame, translation/rotation or waypoints, maximum duration, and observable completion conditions. The same GPT snapshot reviews the candidate against the original proposal and scene in a separate call. A deterministic interface check independently validates types, finite values, action bounds, and supported calls. Allow at most a preregistered number of revisions. Rejected or expired corrections are not silently executed, and invalid proposals count against completion and cost. GPT approval is not a simulator success verdict.

3. Three Technical Approaches to the Correction Branch

Approach A: geometric EEF edits. GPT predicts a small translation and rotation offset for a selected VLA segment, with explicit tool- or world-frame semantics. Apply the edit to a trajectory reference and preserve the rest of the valid proposal. This is the simplest interface for testing whether the reviewer can identify misalignment and useful correction direction. Compare with matched fixed or perception-derived offsets; correct units and orientation composition are essential.

Approach B: bounded Code-as-Policy repair. GPT produces a short local program using declared EEF primitives, for example re-observe, move to a corrected approach pose, and resume the skill. The program may use local feedback but has a fixed duration and revision budget. It cannot replace the task with an unrestricted new plan. Retaining the VLA proposal, affected segment, and handoff rule makes the correction inspectable and separates local repair from full GPT takeover.

Approach C: prediction-assisted review. Present GPT with a standardized kinematic preview or observation-conditioned prediction of the original and corrected short motions, including uncertainty. GPT selects or revises the correction using only deployable evidence. Compare with the same reviewer without previews, matching call budgets. A learned preview can be wrong; future simulator states are never shown to the reviewer. This approach tests whether action consequences improve binary judgment beyond inspecting the current image and action numbers.

4. Pipeline, State Validity, and Runtime Boundaries

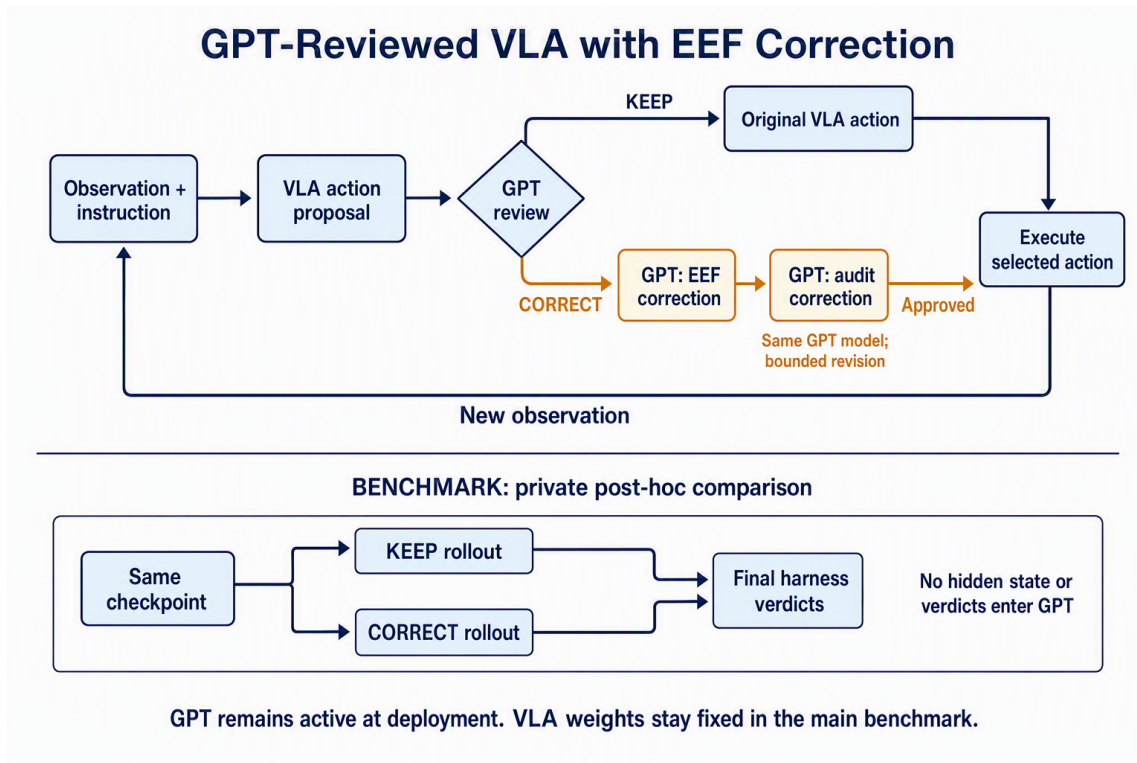


Figure 1: GPT remains in the deployment loop. Review selects KEEP or CORRECT; GPT-generated EEF corrections are audited before execution. The separate benchmark panel compares outcomes privately and never returns hidden state or branch verdicts to GPT. GPT-image-2 conceptual diagram, not a measured result.

One GPT model, several roles. Selection, correction generation, and audit can use the same fixed GPT model and version. They are distinct calls with bounded inputs, not independent sources of evidence. Correlated mistakes and confirmation bias are therefore explicit risks. Compare a single-pass choice with a second-pass audit, while holding total token or call budgets fixed. A model name, including a chosen GPT release, is a configuration to record rather than a guarantee of review accuracy.

Preventing stale edits. Before applying an EEF correction, check the observation and proposal versions. After any intervention, obtain a fresh observation and re-query the VLA; replaying an old, geometrically invalid suffix is prohibited. During review, use a declared hold or validated executable prefix. Environmental evolution and review latency are included in real-time evaluation. A paused-simulation diagnostic must be labeled separately and cannot establish deployment responsiveness.

Existing implementation entry points. Robot-RSI exposes relative EEF motion, pose observation, and servo-based correction paths. These support a concrete adapter but do not establish that arbitrary VLA chunks can already be edited correctly. Validate action scaling, frames, quaternion conventions, gripper semantics, cancellation, and action ownership. Hidden target positions, simulator predicates, and evaluator-owned checkpoints remain outside GPT and generated-code access.

5. Benchmark Design: Selection, Correction, and Full Episodes

Proposed benchmark. We propose *VLA Review Benchmark* as a working title. It evaluates the deployed VLA+GPT system, with frozen VLA and reviewer configurations. The two tracks below distinguish local decision quality from complete robot behavior. The benchmark is initially built on LIBERO short tasks; its checkpoint and disturbance extensions are reported separately from standard task-level scores.

Track A: paired checkpoint continuations. Retain checkpoints from natural VLA rollouts using an evaluator-only mechanism. At a checkpoint, present the same permitted observation and original VLA proposal to each reviewer. Execute the KEEP continuation and the submitted CORRECT continuation in separate simulator branches with matched initial state and exogenous randomness, using the same frozen VLA for subsequent continuation. The agent cannot inspect checkpoint internals or other branch outcomes. Endpoint labels come from final harness verdicts, not a GPT judgment that the EEf pose looks better.

Let $Y_K, Y_C \in \{0, 1\}$ be final success for the unchanged proposal and the submitted correction. The four cases $(0, 1)$, $(1, 0)$, $(1, 1)$, and $(0, 0)$ respectively indicate a beneficial correction, a harmful correction, equal successful outcomes, and unresolved failure. These labels describe the evaluated candidate and continuation, not a universal oracle declaring that a state is correctable by any possible method. For stochastic environments, estimate outcome probabilities over matched continuation seeds rather than relying on one binary result.

Lock the review decision before evaluating either branch. For KEEP decisions, obtain a counterfactual correction in a separate diagnostic call without allowing the decision to change. This avoids measuring correction quality only on states the reviewer selected. Its extra generation and branching costs belong to benchmark construction, not deployed inference cost. Track A isolates checkpoint decisions; Track B includes actual review delay and environmental evolution.

Track B: closed-loop episodes. Deploy the complete reviewer with its real decisions, repeated observations, correction audits, and latency. This tests distribution shift caused by its own interventions. A good checkpoint score does not ensure a good multi-step hybrid policy. Include unperturbed tasks where the VLA already works, local geometric errors that EEf edits may repair, and cases outside the allowed correction envelope. The last group prevents the benchmark from rewarding plausible but ineffective edits; failure remains a legitimate outcome.

Proposed panel and splits. An initial diagnostic test panel contains 24 held-out task-condition pairs with 10 retained checkpoints each: 240 checkpoints. Add a closed-loop panel of the same 24 pairs with five independent initial seeds: 120 episodes per method. Development and validation use different task families and objects. These counts are starting designs, adjusted using development variance; near-duplicate task recipes stay within one split. Release a naturally sampled panel and a separately labeled, stratified stress panel rather than silently balancing outcomes to favor correction.

Configuration and fairness. Freeze model snapshots, prompts, correction bounds, review frequency, revision limits, preview methods, and budgets before testing. Record model-specific sampled checkpoints; do not assume one VLA’s failure states represent another VLA. Compare additional VLA backbones and reviewer models on declared tracks. All algorithms receive the same allowed sensing, action access, and wall-time accounting. The standard 120-task LIBERO panel additionally reports pass@10 by spatial/object/goal and task/object/swap/lan.

6. Planned Experiments and Benchmark Metrics

Main comparisons. Use VLA-only execution, GPT review constrained to KEEP, unconditional GPT EEF correction, a heuristic gate with the same GPT corrector, and the full GPT gate–corrector–audit system. Add a GPT gate with a fixed geometric repair to isolate judgment from generation. An unrestricted GPT takeover is a separate capability/cost reference, not an equal-action-space baseline. Compare geometric edits, bounded programs, and prediction-assisted review under matched budgets.

Decision metrics. On paired continuations, report the rate of choosing CORRECT when $(Y_K, Y_C) = (0, 1)$ and KEEP when $(1, 0)$. Report the unresolved and equal-outcome fractions separately. In equal-success cases, unnecessary intervention incurs cost even when success is preserved. A paired candidate regret can be computed as $\max(Y_K, Y_C) - Y_b$, where Y_b is the selected branch outcome. This is a hindsight diagnostic, never an online selection signal or proof of optimality beyond those two candidates.

Correction metrics. Measure accepted program validity, actual EEF displacement and orientation change, correction duration, local recovery evidence, and final success change. Report harm on checkpoints where the original VLA succeeds. Rejected candidates, audit revisions, correction-limit saturation, and unexecuted proposals remain visible. Improvement from moving the EEF closer to a visually chosen point is not sufficient if it reduces final task success.

Experiment A: preserve versus rescue. On the natural panel, compare the hybrid’s paired success gain and intervention rate against the VLA. On the stress panel, separate local misalignment, grasp/placement mistakes, observation ambiguity, and failures beyond the correction envelope. A useful system should rescue some VLA failures while limiting newly introduced failures. Report both terms instead of only net improvement.

Experiment B: is audit useful? Compare no second-pass audit, same-model audit, and deterministic interface checks alone. Hold total model budgets fixed in one comparison and allow actual incurred costs in another. Evaluate whether the second pass catches bad edits or merely repeats the original rationale. GPT self-consistency is not independent verification; only executed outcomes establish intervention benefit.

Experiment C: correction scope and state freshness. Vary EEF edit magnitude, orientation freedom, repair duration, and revision count. Compare fresh re-observation and VLA re-query after edits with a labeled diagnostic that reuses the old proposal suffix. Separately inject review delay to quantify stale-action failure. This paper uses latency as a cost and failure factor; designing a new low-latency scheduler remains the second proposal’s objective.

Possible result	Defensible interpretation
More rescued failures than damaged successes, with bounded cost	Supports selective deployed correction over the frozen VLA.
Always-correct performs equally well	Binary review has not established an independent advantage.
Audit raises call cost without reducing harmful edits	The extra review pass is not justified under the measured budget.
Checkpoint gains disappear in closed-loop episodes	Intervention-induced state shift or stale handoffs remain unresolved.
Small geometric edits match generated programs	Simple correction may suffice; narrow the Code-as-Policy claim.

7. Statistical Protocol, Risks, and Expected Contribution

Primary evaluation. Preregister closed-loop task-success improvement at a fixed review/correction budget, with the harmful-intervention rate on baseline-successful checkpoints as a key secondary endpoint. Use paired seeds and intervals clustered by underlying task; repeated checkpoints from one rollout are not independent tasks. Report uncertainty for candidate-pair labels and candidate regret. Never return test-branch outcomes to the reviewer or use them to tune its gate. Benchmark construction and test outcome assignment remain evaluator-side operations.

Efficiency and reliability. Report prompt/completion/total tokens, VLM calls and unmetered calls, review latency, EEf correction time, recovery time, and total wall time per episode. Compare success–intervention–cost curves rather than assuming every extra review is valuable. Distinguish natural sampling from any balanced stress set. Count success only from the final simulator or independent harness. Provider, transport, image, resource, and interrupted records are excluded from valid-verdict task denominators and reported separately; implementation errors, rejection exhaustion, and missing coverage are also reported.

Principal failure modes. A reviewer can hallucinate a geometric error, confuse coordinate frames, or overrule a correct VLA. The same GPT used for generation and audit can reinforce its own mistake. A bounded correction can be locally plausible but incompatible with the next policy state. Sensing ambiguity can make reliable branch selection impossible. Each failure class needs retained images, visible rationale, action receipts, and final verdicts; hidden reasoning and simulator state are never exposed to the evaluated agent. Preserve original, corrected, failed, rejected, and diagnostic runs.

Why this is a benchmark contribution. Existing task success alone cannot reveal whether a reviewer knew when to intervene, whether its correction helped, or whether extra calls merely compensated for a poor base policy. Paired candidate continuations plus complete hybrid rollouts expose these distinctions. The benchmark should report candidate-dependent labels, natural prevalence, cross-VLA transfer, and latency conditions. It must not present privileged hindsight selection as a deployable competitor.

Optional learning extension. Correction traces may later support DAgger-style aggregation or VLA fine-tuning. If studied, evaluate the updated VLA *with GPT still active* under the same deployed architecture, and separate training gains from runtime correction gains. A GPT-free ablation can quantify component contribution, but removing GPT is neither the objective nor a requirement of this proposal. The main paper can be completed with frozen policy weights.

Expected paper claim. A persistent GPT reviewer can selectively retain VLA actions or improve them through audited EEf corrections, with measurable gains and identifiable limits. The method contribution is the binary review-and-correction interface; the experimental contribution is a benchmark that separates preserving correct behavior, repairing mistakes, and unnecessary or harmful intervention. This claim is independent of the first paper’s control representation and the second paper’s execution-timing mechanism.

References

- [1] Code as Policies: Language Model Programs for Embodied Control. 2022. <https://arxiv.org/abs/2209.07753>
- [2] π_0 : A Vision-Language-Action Flow Model for General Robot Control. 2024. <https://arxiv.org/abs/2410.24164>
- [3] Real-Time Execution of Action Chunking Flow Policies. 2025. <https://arxiv.org/abs/2506.07339>
- [4] Ross, Gordon, and Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. 2011. <https://arxiv.org/abs/1011.0686>
- [5] Noematrix Team. RoboRSI research preview. 2026. <https://lab.noematrix.ai/blog/2-roborsi-research-preview/>